The Basics of Lexical Analysis
Writing a State-Machine Lexer

CS 331 Programming Languages

Lecture Slides

Wednesday, February 8, 2023

Glenn G. Chappell

Department of Computer Science

University of Alaska Fairbanks

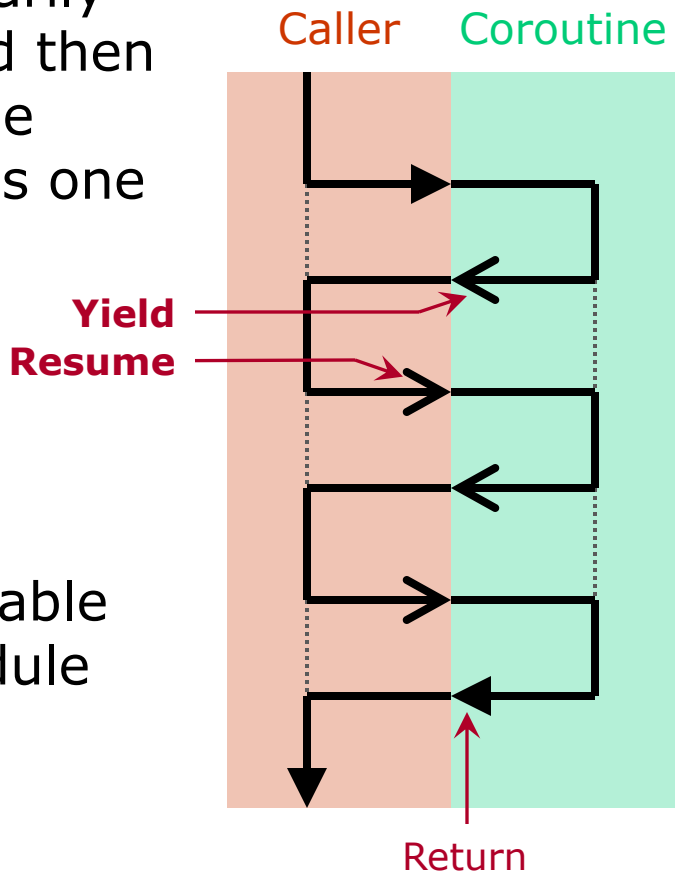
ggchappell@alaska.edu

© 2017–2023 Glenn G. Chappell

Review

Review
Lua: Advanced Flow — Coroutines

A **coroutine** is a function that can temporarily give up control (**yield**) at any point, and then later be **resumed**. Each time a coroutine temporarily gives up control, it may pass one or more values back to its caller (it **yields** these values).



Coroutines are available in a number of programming languages. They are available in Lua through the standard-library module `coroutine`.

See `adv.lua`.

Review
Lua: Advanced Flow — Custom Iterators

A Lua for-in loop uses an **iterator**. We can write our own function returning a custom iterator as follows.

If needed, create variables here to store info between calls to `iter`.

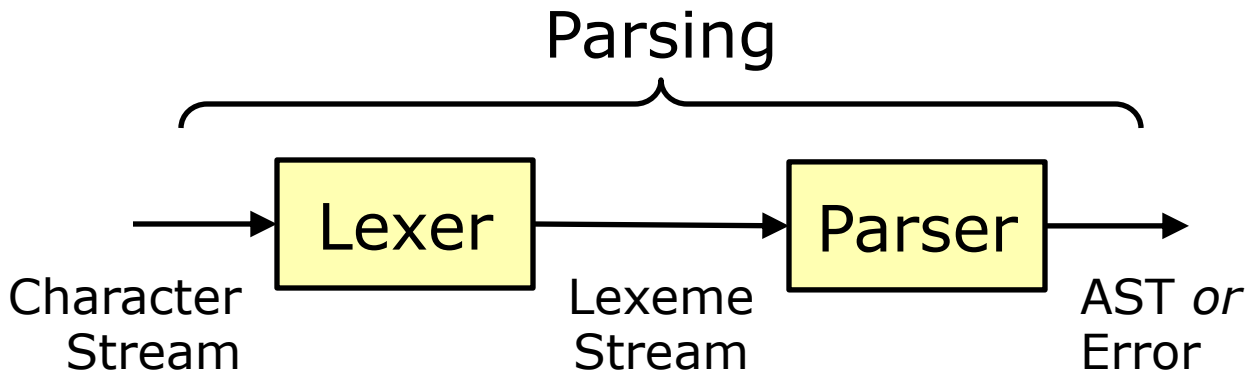
See `adv.lua`.

```
function FUNCNAME(…)
    local …
    local function iter(dummy1, dummy2)
        if … then
            return nil  -- Iterator exhausted
        end
        …
        return …        -- Next value(s)
    end
    return iter, nil, nil
end
```

To *use* this, do something like

```
for k in FUNCNAME(…)
    …
end
```

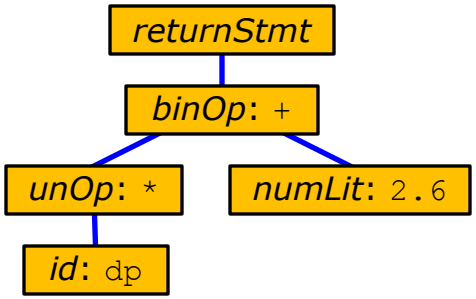
Review
Overview of Lexing & Parsing



`return (*dp + 2.6); //x`

`return (*dp + 2.6); //x`

key, op, id, op, num lit, punct, punct



Two phases:

- **Lexical analysis** (**lexing**)
- **Syntax analysis** (**parsing**)

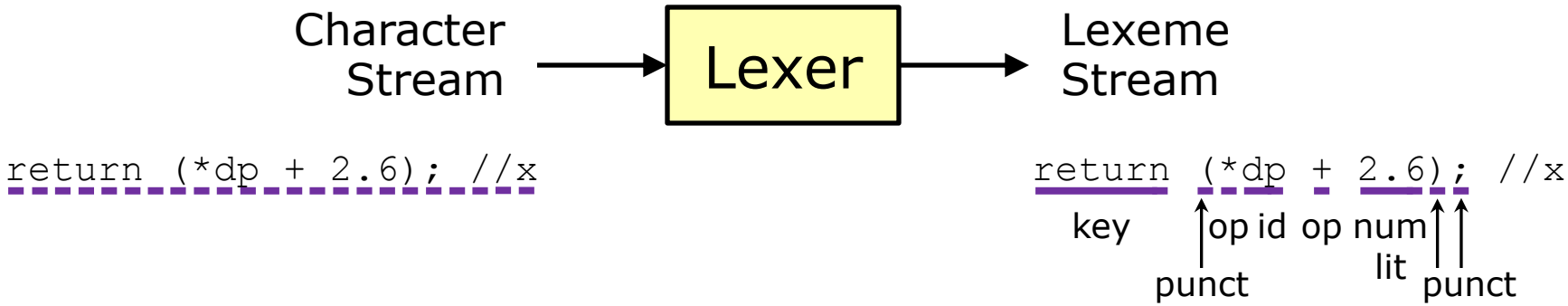
The output of a parser is typically an **abstract syntax tree** (**AST**). Specifications of these will vary somewhat.

The Basics of Lexical Analysis

The Basics of Lexical Analysis
Introduction

Now we look closer at the first phase of parsing: lexical analysis. Shortly, we will begin discussing how to write a lexer.

As we have said, a **lexical analyzer**, or **lexer**, reads a character stream and outputs a lexeme stream.



Each lexeme is typically classified as belonging to a particular **category**. Next we look at some common lexeme categories.

The Basics of Lexical Analysis
Lexeme Categories [1/5]

An **identifier** is a name that a program gives to some entity: variable, function, type, namespace, etc.

In the C++ code below, the identifiers are circled.

```
class MyClass {
public:
    void myFunc(Type1 & aa, int bb) const
    {
        for (int ii = -37; ii <= bb; ++ii)
        {
            aa.foo();
        }
    }
};
```

The Basics of Lexical Analysis
Lexeme Categories [2/5]

A **keyword** is an identifier-looking lexeme that has special meaning within a programming language.

In the C++ code below, the keywords are circled.

```
class MyClass {
public:
    void myFunc(Type1 & aa, int bb) const
    {
        for (int ii = -37; ii <= bb; ++ii)
        {
            aa.foo();
        }
    }
};
```

The Basics of Lexical Analysis
Lexeme Categories [3/5]

An **operator** is a word that gives an alternate method for making something like a function call. The arguments of an operator are called **operands**.

In the following code, the operators are "`+=`", "`*`", and "`-`". The operands of "`+=`" are "`aaa`" and "`b * -c`".

```
aaa += b * -c;
```

The **arity** of an operator is the number of operands it has.

- A **unary** operator has one operand, like "`-`" in the above code.
- A **binary** operator has two operands, like "`+=`" and "`*`" in the above code. A binary operator that is placed between its operands is an **infix** operator.
- A **ternary** operator has three operands. Ternary operators are uncommon. For example, Lua does not have any. However, C++ and Java have one: "… `?` … `:` …".

The Basics of Lexical Analysis
Lexeme Categories [4/5]

A **literal** is a bare value in code.

Below are some literals in C++ and Lua.

C++ Literals	
Literal	**Type**
`42`	`int`
`42.5`	`double`
`42.5f`	`float`
`false`	`bool`
`'A'`	`char`
`"zebra"`	`char[]`

Lua Literals	
Literal	**Type**
`42.5`	`number`
`false`	`boolean`
`"zebra"`	`string`
`[=[xy]=]`	`string`
`{ 1, 2 }`	`table`
`nil`	`nil`

In some cases a literal might not be a single lexeme. The Lua table literal shown here is an example; it consists of 5 lexemes.

The Basics of Lexical Analysis
Lexeme Categories [5/5]

Punctuation is the category for the extra lexemes in a program that do not fit into any of the previously mentioned categories.

Punctuation in C++ includes braces (`{ }`), semicolons (`;`), and the colon (`:`) after `public`, `private`, or the cases in a switch-statement.

Lexeme categories mentioned:

- **Identifier**
- **Keyword**
- **Operator**
- **Literal**
- **Punctuation**

The Basics of Lexical Analysis
Reserved Words [1/3]

A **reserved word** is a word that fits the general specification of an identifier, but is not allowed as a legal identifier in a program.

Note that, while this is an important concept, *reserved word* is not a lexeme category.

In many programming languages, the keywords and the reserved words are the same.

However, one can imagine a variant of (say) C in which the compiler could distinguish how a word is used, based on its position in the code. Then there could be keywords that are not reserved words. Something like the following might be legal.

```
for (for for = 10; for; --for) ;
```

The Basics of Lexical Analysis
Reserved Words [2/3]

Since the 2011 Standard, C++ has had keywords that are not reserved words; one of these is `override`. This has special meaning when placed after the parentheses in a member-function declaration, but otherwise it is a legal identifier.

So the following is legal C++, by every standard since 2011.

```
class Derived : public Base {
    virtual void override() override;
        // Derived member function named "override"
        // Overrides Base member function "override"
    …
```

The Basics of Lexical Analysis
Reserved Words [3/3]

The programming language Fortran traditionally has no reserved words. The following is, famously, legal code in at least some versions of Fortran.

```
IF IF THEN THEN ELSE ELSE
```

On the other hand, there can be reserved words that are not keywords. The Java standard specifies that `const` and `goto` are reserved words. However, neither is a keyword. Thus, these words cannot legally be included in a Java program at all.

Note. We will use our definitions consistently in this class. Be aware, however, that some people use the term *keyword* to mean what we mean by *reserved word*. (For example, the C++ Standard does not refer to `override` as a keyword.)

The Basics of Lexical Analysis
Lexer Operation

A lexer outputs a sequence of lexemes. There is usually no need to store these lexemes. Rather, the lexer can provide get-next-lexeme functionality, which the parser can then use.

There are essentially three ways to write a lexer.

- State machine, entirely in code.
- State machine that uses a table (I do *not* mean a Lua table).
- Using a more advanced method, suitable for writing a parser.

Various software packages—e.g., `lex`—automatically generate code for a lexer, given regular expressions describing the lexeme categories. These use one of the above three methods.

We will write a lexer using the first method. When we are done, it should be easy see how we might have used a table instead.

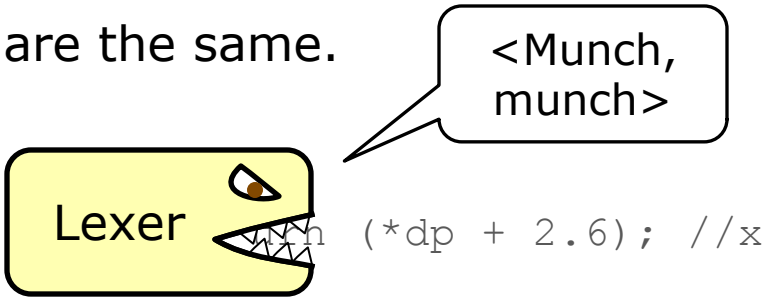
Writing a State-Machine Lexer

Writing a State-Machine Lexer
Introduction [1/3]

We wish to write a lexer, in Lua, for a hypothetical PL. Our lexer will be an entirely hand-coded state machine.

Lexemes are described in the *In-Class Lexeme Specification*.

- Identifiers are essentially as in C/C++.
- All whitespace will usually be treated the same.
- Lexemes may be arbitrarily long. *Maximal munch** rule applies.
- There might not be any delimiter between lexemes.
- Comments are like multiline C/C++ comments.
- The keywords and reserved words are the same.



*The **maximal munch** rule says that a lexeme is always the longest substring beginning from its starting point that can be interpreted as a lexeme.

Writing a State-Machine Lexer
Introduction [2/3]

Note that our rules for lexemes are not universal for all PLs. For example:

- Haskell has different rules for identifiers.
- In Python, Haskell, JavaScript, and Go, a newline may serve as something like an end-of-statement marker.
- In Forth, consecutive lexemes are *always* separated by whitespace.
- Lua uses different syntax for comments.

Writing a State-Machine Lexer
Introduction [3/3]

A skeleton for a lexer is in `lexer.lua`. Our task is to finish this file, turning it into a complete lexer for a hypothetical programming language, whose lexical structure is specified in the *In-Class Lexeme Specification*.

In Assignment 3 you will write a similar lexer, based on a different lexeme specification. This lexer will eventually become part of an interpreter for an actual (not hypothetical) programming language.

I have posted a simple program that uses `lexer.lua`, passing a string and printing the lexemes generated.

See `uselexer.lua`, `lexer.lua`.

Writing a State-Machine Lexer Design [1/4]

We will write a Lua module `lexer` with the following interface.

- The module has a function `lexer.lex`, which is given a string (`program`) and returns an iterator that goes through lexemes.
- The iterator returns pairs: string and number. The string is the lexeme itself. The number represents the lexeme category.
- The category is an index for table `lexer.catnames`, whose values will be human-readable string forms of the category names.

So the following prints text & category of all lexemes in `program`.

```
lexer = require "lexer"

for lexstr, cat in lexer.lex(program) do
    local catstr = lexer.catnames[cat]
    io.write(string.format("%-10s  %s\n",
                           lexstr, catstr))
end
```

Writing a State-Machine Lexer Design [2/4]

The function returned by `lexer.lex` will be a closure. Whatever information is necessary for lexing will be stored in this closure. (Such information is the kind of thing we might store in data members in a C++ object.)

Internally, our lexer will run as a **state machine**.

- A state machine has a current **state**. This might simply be a number. Code that runs as a state machine will need to store this.
- The machine proceeds in a series of steps. At each step, it looks at the current item (here, character) in the input and the current state. It then decides what state to go to next.
- Based on the state and current character, our state machine may also make other decisions. Examples might include declaring a lexeme to be complete, or setting the category of a lexeme.

Writing a State-Machine Lexer Design [3/4]

Variables

- The input is the given string: `program`.
- The index of the next character to read is stored in variable `pos` (which starts at `1`, since this is Lua).
- The state is stored in variable `state`, initialized as `START`.
- We build a lexeme in string `lexstr`, initialized as empty (`""`).
- The category of a complete lexeme is stored in `category`.

When a complete lexeme has been found, set `state` to `DONE`, and set `category` appropriately (`lexer.ID`, `lexer.KEY`, etc.).

Writing a State-Machine Lexer Design [4/4]

Utility Functions

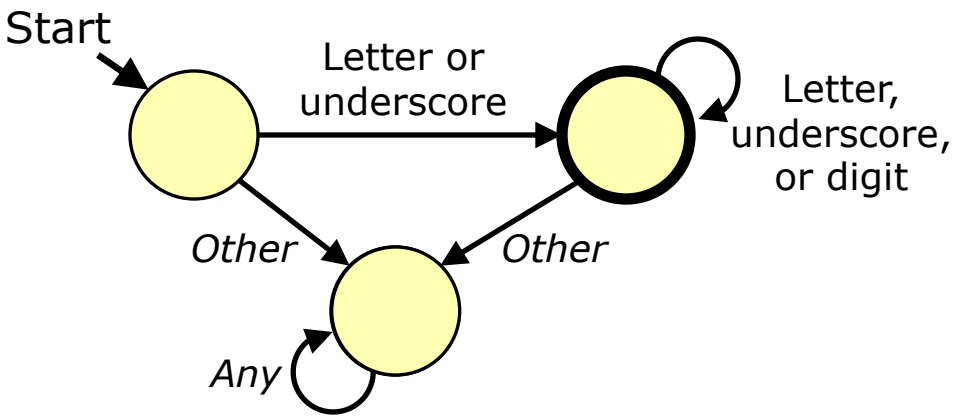
- To add the current character to the lexeme, call `add1()`.
- To skip the current character without adding it, call `drop1()`.
- Lua has no character type. We represent a character as a string of length one. I have written character-testing functions (`isLetter`, `isDigit`, `isWhitespace`, `isIllegal`). Each takes a string. When given a string whose length is not exactly one, each returns `false`.

Writing a State-Machine Lexer
Coding a State Machine I [1/4]

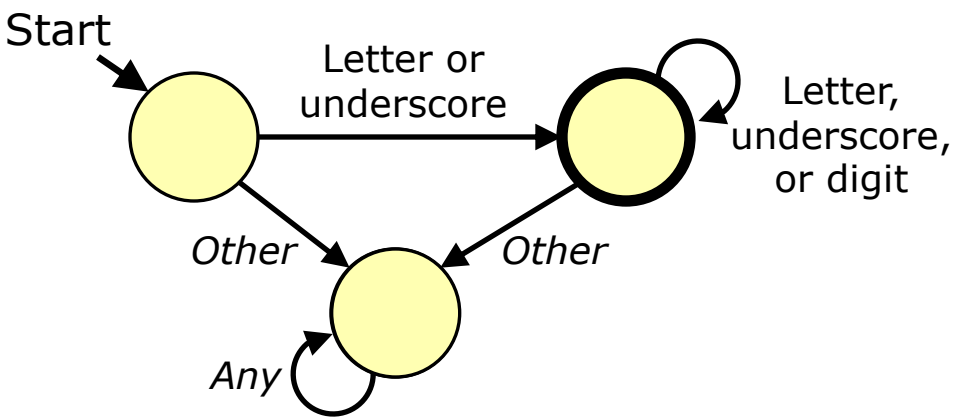
Let's handle **Identifiers** and **Keywords**—ignoring the distinction between the two, for the moment.

An **Identifier-or-Keyword** begins with a letter or underscore (_). Following this are zero or more characters, each of which is a letter, underscore, or digit.

Here is a DFA that recognizes an **Identifier-or-Keyword**.



Writing a State-Machine Lexer
Coding a State Machine I [2/4]

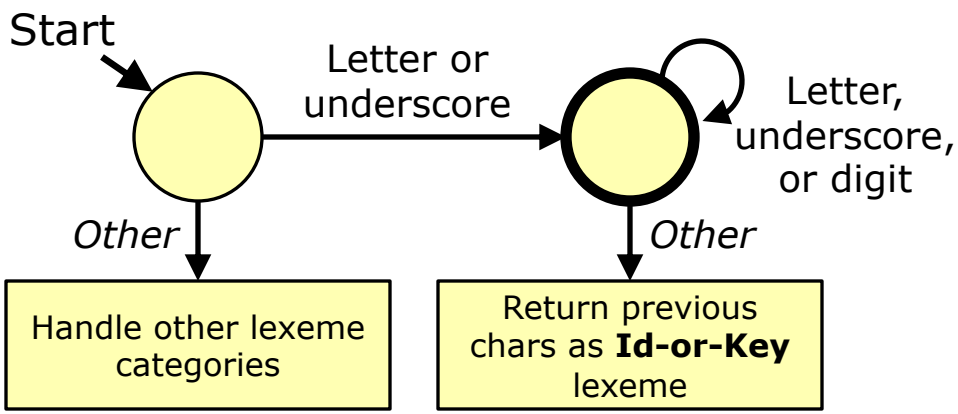


The above DFA is not quite appropriate for what we need to do.

- It determines whether the *entire input* is an **Identifier-or-Keyword**. But we need find these followed by other lexemes.
- We need to recognize other categories of lexemes as well.

Note that the two transitions to the lower state are qualitatively different. For our purposes, the one on the left indicates that we have a different lexeme category. The one on the right indicates that an **Identifier-or-Keyword** lexeme is complete.

Writing a State-Machine Lexer
Coding a State Machine I [3/4]



Here is an almost-DFA that better expresses our process.

- The right-hand box is the `DONE` state mentioned earlier.
- The left-hand box involves other states that we will cover later.
- The left-hand circle is the `START` state.

I like to name each new state using a short string that gets the state machine into it. I will call the right-hand circle "`LETTER`". Note that this does *not* mean that we have just read a letter.

Writing a State-Machine Lexer
Coding a State Machine I [4/4]

How do we differentiate between **Identifier** and **Keyword** lexemes? We could use the state-machine idea, but the code would be complicated.

An easier method: when an **Identifier-or-Keyword** lexeme is complete, compare it to each **Keyword**, and set the category appropriately.

TO DO

- Write code to handle both **Identifier** and **Keyword** lexemes.
- Test to see whether this code works.

Done. See `lexer.lua`.

Writing a State-Machine Lexer
TO BE CONTINUED …

Writing a State-Machine Lexer will be continued next time.